HANIKA ANGLICAN INTEGRATED POLYTECHNIC (HAIP) · Data Mining and Data Warehousing · FA1 Group Work

Security Implementation

Authentication, RBAC enforcement, masking, encryption and audit logging - with test results

---

Project: University Library Analytics Data Warehouse

Group: **Group X** (replace with your group number)

Members: Member 1 (Data Architect) · Member 2 (ETL Specialist) · Member 3 (Analytics & Security Lead)

Database: SQLite 3.53.1 · ETL: Python 3 + pandas · Dashboards: Power BI Desktop

Submission: November 2, 2026 · Presentation: November 6, 2026

Contents

1. Overview
2. Authentication
3. RBAC enforcement with the SQLite authorizer
4. Data masking
5. Row-level security for department heads
6. Encryption
7. Audit logging
8. Test results

1. Overview

SQLite is a file database without user accounts, so security is implemented in **four layers**:

Layer	What	Where
1. File	only the admin, ETL and backup accounts have Windows permissions on `library_dw.db`; the server disk is encrypted with BitLocker	Windows / NTFS
2. Database	roles, permissions and users stored as tables; masked view; department views; audit table and triggers	`06_security_rbac.sql`, `07_audit_logging.sql`
3. Access program	login with hashed passwords, lock-out, **SQLite authorizer** that checks every column read and every change, private department tables, audit of every query	`security_tools/secure_access.py`
4. Reports	Power BI receives only masked data; Row-Level Security role for department heads	`dashboard_guide.pdf` section 6

2. Authentication

- Users are stored in `sec_users` with a **PBKDF2-SHA256** hash (200,000 iterations) and a random 16-byte salt per user - the clear password is never stored.
- Comparison with `secrets.compare_digest` (constant time).
- Wrong username and wrong password give the **same** message, so an attacker cannot discover valid usernames.
- **3 failed attempts → account locked for 24 hours** (`failed_logins`, `locked_until`).
- Password age is checked at login (90 days).

```
python secure_access.py --setup          # create users (hashes only)
python secure_access.py --user head_cs   # log in
```

3. RBAC enforcement with the SQLite authorizer

SQLite calls a Python function **before every step of every query** ("may this user read column X of table Y?", "may this user DELETE?"). `secure_access.py` answers with *OK* or *DENY* according to the role:

Role	Authorizer rule
dw_admin	everything allowed
dw_etl	everything except `sec_*` tables and `audit_log`
dw_backup	read only, no `sec_*` tables through the program
dw_analyst	only SELECT; may read `dim_*` and `fact_*` (and views on them); **reading `dim_student.student_id` is denied**; staging, security and audit tables denied; INSERT/UPDATE/DELETE/CREATE/DROP/PRAGMA/ATTACH denied
dw_dept_head	only SELECT on the private `my_*` tables of the session; everything else denied

In addition, read-only roles open the database file in **read-only mode** (`mode=ro`), so even a mistake in the rules could not change data.

4. Data masking

- At load time the ETL creates `student_id_masked`: `STU-2024-001` → `STU-****-001` (the intake year is hidden, the number keeps students distinguishable for counting).
- The real `student_id` column is blocked for analysts by the authorizer (column level) and never exported to Power BI (`dim_student.csv` has no real ID; ODBC users connect to `v_dim_student_masked`).
- Department views (`v_dept_*`) contain only the masked ID.

5. Row-level security for department heads

At login of a department head the program copies **only that department's rows** from the views `v_dept_book_loans`, `v_dept_digital_usage`, `v_dept_room_bookings` and `v_dept_monthly_summary` into private temporary tables `my_book_loans`, `my_digital_usage`, `my_room_bookings`, `my_monthly_summary`. These tables exist only in the memory of that session and disappear at logout. The authorizer allows the head to read these tables and nothing else - not the facts, not the department views of other departments.

6. Encryption

Data	Protection
At rest - database file	BitLocker (AES) on the disk of the library server; NTFS permissions limited to admin + service accounts. For per-file encryption, the open-source SQLCipher build of SQLite can be used without changing the model.
At rest - backups	encrypted with AES (Fernet: AES-128-CBC + HMAC-SHA256) by `backup_restore.py full --encrypt`; the key is stored separately from the backups (USB in the Director's safe)
In transit	the database is not exposed on the network. Reports travel through the Power BI service over **HTTPS/TLS 1.2+**; the on-premises gateway connects outbound over TLS; file transfers to the backup share use SMB 3 encryption
Passwords	PBKDF2-SHA256 hashes only

7. Audit logging

What is recorded	How
successful and failed logins, lock-outs	`secure_access.py` → `audit_log` (LOGIN_OK, LOGIN_FAILED)
every query - allowed (QUERY + rows returned) and refused (DENIED + reason)	`secure_access.py` → `audit_log`
UPDATE / DELETE of student data and facts	**triggers** `trg_dim_student_update`, `trg_dim_student_delete`, `trg_fact_loan_update`, `trg_fact_loan_delete`, `trg_fact_booking_delete`
ETL runs	`etl_batch_log` (+ every data problem in `etl_error_log`)

While an ETL batch is RUNNING the triggers stay silent (the reload is already recorded in `etl_batch_log`), which avoids thousands of useless audit rows every night. Review with `SELECT * FROM v_audit_report;`.

8. Test results

The command `python secure_access.py --demo` runs the security tests. Output of the last run:

```
================================================================================
 SECURITY TESTS - library_dw.db
================================================================================

[1] ANALYST (analyst_amina): read-only, masked StudentID
  OK       SELECT COUNT(*) FROM fact_book_loan
           -> ['COUNT(*)'] [(2205,)]
  OK       SELECT student_id_masked, student_level FROM dim_student LIMIT 3
           -> ['student_id_masked', 'student_level'] [('STU-****-001', 'Undergraduate'),
('STU-****-002', 'Undergraduate'), ('STU-****-003', 'Undergraduate')]
  REFUSED  SELECT student_id FROM dim_student          (real ID)
           -> access to dim_student.student_id is prohibited
  REFUSED  SELECT * FROM dim_student                   (includes real ID)
           -> access to dim_student.student_id is prohibited
  REFUSED  DELETE FROM fact_book_loan WHERE loan_key = 1
           -> not authorized
  REFUSED  SELECT * FROM stg_book_transactions         (staging area)
           -> access to stg_book_transactions.stg_id is prohibited
  REFUSED  SELECT * FROM sec_users                     (password hashes)
           -> access to sec_users.username is prohibited

[2] DEPARTMENT HEAD (head_cs): only own department
  OK       SELECT department_code, COUNT(*) FROM my_book_loans GROUP BY 1
           -> ['department_code', 'COUNT(*)'] [('CS', 374)]
  OK       SELECT month_label, book_loans, digital_downloads FROM my_monthly_summary
           -> ['month_label', 'book_loans', 'digital_downloads', 'room_bookings'] [('2025-
12', 8, 106, 13), ('2025-11', 5, 174, 21), ('2025-10', 12, 176, 12)] ...
  OK       SELECT student, title FROM my_book_loans LIMIT 2
           -> ['student', 'title'] [('STU-****-092', 'Software Engineering in Practice:
Advanced Readings'), ('STU-****-050', 'Computer Networks Explained: Advanced Readings')]
  REFUSED  SELECT COUNT(*) FROM fact_book_loan          (all departments)
           -> not authorized
  REFUSED  SELECT COUNT(*) FROM v_dept_book_loans       (all departments)
           -> access to dim_department.department_code is prohibited

[2] DEPARTMENT HEAD (head_eng): only own department
  OK       SELECT department_code, COUNT(*) FROM my_book_loans GROUP BY 1
           -> ['department_code', 'COUNT(*)'] [('ENG', 404)]
  OK       SELECT month_label, book_loans, digital_downloads FROM my_monthly_summary
           -> ['month_label', 'book_loans', 'digital_downloads', 'room_bookings'] [('2025-
12', 9, 137, 9), ('2025-11', 16, 259, 19), ('2025-10', 9, 174, 23)] ...
  OK       SELECT student, title FROM my_book_loans LIMIT 2
           -> ['student', 'title'] [('STU-****-092', 'Thermodynamics for Engineers:
Advanced Readings'), ('STU-****-019', 'Civil Engineering Drawing: Advanced Readings')]
  REFUSED  SELECT COUNT(*) FROM fact_book_loan          (all departments)
           -> not authorized
  REFUSED  SELECT COUNT(*) FROM v_dept_book_loans       (all departments)
           -> access to dim_department.department_code is prohibited

[3] WRONG PASSWORD 3 TIMES (head_bus) -> account locked
  attempt 1: Wrong username or password
  attempt 2: Wrong username or password
  attempt 3: Wrong username or password - account is now LOCKED
  correct password now: Account locked until 2026-10-27T19:38:56
  (admin unlocked head_bus again for the next demo)

[4] ADMIN (lib_admin): a change is recorded by the audit trigger
  OK       UPDATE dim_student SET valid_to = '9999-12-30' WHERE student_key = 2
           -> [] []
  OK       UPDATE dim_student SET valid_to = '9999-12-31' WHERE student_key = 2  (undo)
           -> [] []

[5] AUDIT LOG - the last 12 events
```

```
  2026-10-26 19:38:56 lib_admin      QUERY                     UPDATE dim_student SET
valid_to = '9999-12-31' WHERE student_key = 2
  2026-10-26 19:38:56 direct database access UPDATE        dim_student  level:
Undergraduate -> Undergraduate; department: SCI -> SCI; valid_t
  2026-10-26 19:38:56 lib_admin      QUERY                     UPDATE dim_student SET
valid_to = '9999-12-30' WHERE student_key = 2
  2026-10-26 19:38:56 direct database access UPDATE        dim_student  level:
Undergraduate -> Undergraduate; department: SCI -> SCI; valid_t
  2026-10-26 19:38:56 lib_admin      LOGIN_OK
  2026-10-26 19:38:56 head_bus       LOGIN_FAILED              account locked
  2026-10-26 19:38:56 head_bus       LOGIN_FAILED              wrong password (3/3) ->
ACCOUNT LOCKED
  2026-10-26 19:38:56 head_bus       LOGIN_FAILED              wrong password (2/3)
  2026-10-26 19:38:56 head_bus       LOGIN_FAILED              wrong password (1/3)
  2026-10-26 19:38:56 head_eng       DENIED                    SELECT COUNT(*) FROM
v_dept_book_loans  ->  access to dim_department.d
  2026-10-26 19:38:56 head_eng       DENIED                    SELECT COUNT(*) FROM
fact_book_loan  ->  not authorized
  2026-10-26 19:38:56 head_eng       QUERY                     SELECT student, title FROM
my_book_loans LIMIT 2
```

Test	Expected	Result
analyst reads facts	allowed	OK
analyst reads masked ID	allowed	OK
analyst reads real `student_id` / `SELECT *` on dim_student	refused	REFUSED
analyst deletes a fact row	refused	REFUSED
analyst reads staging / password table	refused	REFUSED
head_cs reads own department	only CS rows	OK - CS only
head_eng reads own department	only ENG rows	OK - ENG only
head reads all facts / another department's view	refused	REFUSED
3 wrong passwords	account locked	LOCKED
admin changes a student row	recorded by trigger	2 audit rows